CODE:

```
import itertools


def travelling_salesman(graph, start):
    # graph: adjacency matrix (2D list)
    n = len(graph)
    vertices = list(range(n))
    vertices.remove(start)

    min_path = None
    min_cost = float("inf")

    # Try all permutations of cities
    for perm in itertools.permutations(vertices):
        path = [start] + list(perm) + [start]
        cost = sum(graph[path[i]][path[i+1]] for i in range(len(path)-1))
        if cost < min_cost:
            min_cost = cost
            min_path = path

    return min_path, min_cost

# Example graph (symmetric distances)
graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
```

]

path, cost = travelling_salesman(graph, 0)

print("Optimal Path:", path)

print("Total Cost:", cost)

OUTPUT:

```
============ RESTART: C:/Users/arunp/OneDrive/Documents/AI/EXP 9.py =====
Optimal Path: [0, 1, 3, 2, 0]
Total Cost: 80
```